CMPE 125 – Lab 6 Report

Vending Machine FSM with Test Bench

Name:	Yixu Liu	**Student ID:**	017406857
Instructor:	Jahan Ghofraniha	**Date:**	October 28, 2026

1. Design and Calculations

1.1 Operation

The vending machine sells a beverage for 25 cents and accepts a nickel (5 cents), a dime (10 cents), or a quarter (25 cents). Its five states store the current balance. The controller uses Mealy transition logic: the present balance and accepted coin determine the next state and transaction outputs. A completed purchase returns the balance to zero and provides the required change.

For a balance b and an accepted coin worth c cents,

$$b_{\text{next}} = \begin{cases} b + c, & b + c < 25, \\ 0, & b + c \geq 25, \end{cases} \qquad \text{change} = b + c - 25 \quad \text{when } b + c \geq 25.$$

For example, $5 + 25 = 30$ cents returns a nickel, $10 + 25 = 35$ cents returns a dime, and $20 + 25 = 45$ cents returns two dimes. The largest stored balance is 20 cents.

1.2 State Encoding

Five states require $\lceil \log_2 5 \rceil = 3$ state bits, denoted A, B, C. The code names below match the supplied Verilog.

State	Balance (cents)	Encoding ABC	`StateLED[4:0]`
S_0	0	000	00001
S_5	5	001	00010
S_{10}	10	010	00100
S_{15}	15	011	01000
S_{20}	20	100	10000

1.3 State Transition Table

Let N, D, Q denote accepted nickel, dime, and quarter pulses. The input order is (N, D, Q). The output order is (P, R_5, R_{10}, R_{20}) = (Dispense, ReturnNickel, ReturnDime, ReturnTwoDimes).

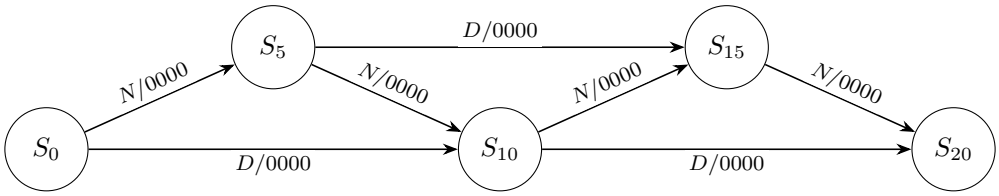
Present state (ABC)	Nickel (100)	Dime (010)	Quarter (001)
S_0 (000)	S_5 (001) / 0000	S_{10} (010) / 0000	S_0 (000) / 1000
S_5 (001)	S_{10} (010) / 0000	S_{15} (011) / 0000	S_0 (000) / 1100
S_{10} (010)	S_{15} (011) / 0000	S_{20} (100) / 0000	S_0 (000) / 1010
S_{15} (011)	S_{20} (100) / 0000	S_0 (000) / 1000	S_0 (000) / 1110
S_{20} (100)	S_0 (000) / 1000	S_0 (000) / 1100	S_0 (000) / 1001

Each entry gives the next state (encoded value) / transaction output.

1.4 State Transition Diagram

The diagram is split into two views of the same FSM for readability. Each arrow is labeled *coin/output*, using the four-bit output order defined on page 1. Reset selects S_0. Without a valid coin, the stored state and registered outputs hold their previous values.

(a) Balance accumulation: total below 25 cents



(b) Purchase completion: dispense and return to zero

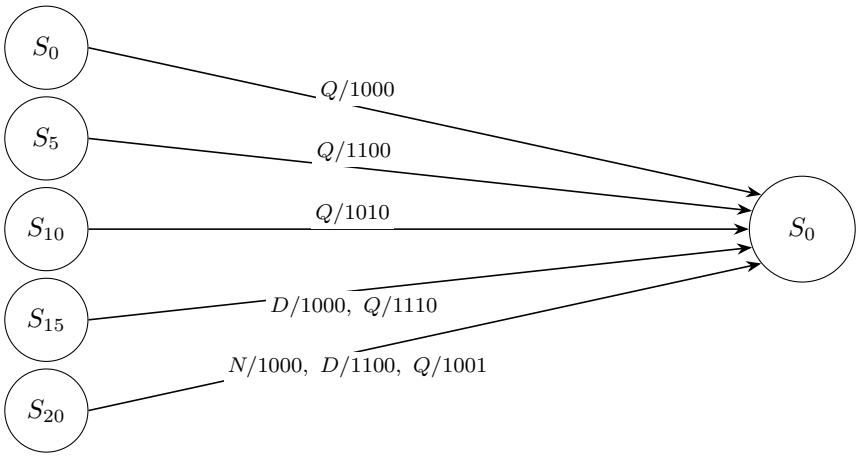


Figure 1: Mealy transition logic for accumulation and dispensing.

1.5 Next-State and Output Logic

Define state indicators from the current state bits:

$$E_0 = \overline{A}\,\overline{B}\,\overline{C}, \qquad E_5 = \overline{A}\,\overline{B}C, \qquad E_{10} = \overline{A}B\overline{C},$$
$$E_{15} = \overline{A}BC, \qquad E_{20} = A\overline{B}\,\overline{C}.$$

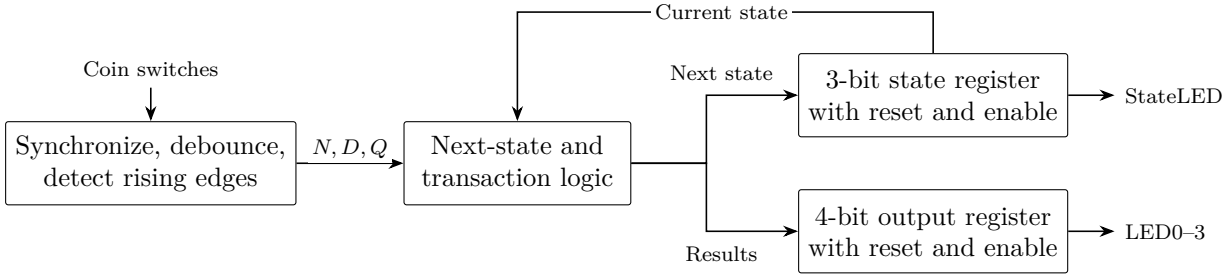
For one valid coin pulse, the transition table gives the following equations. Here + means OR and multiplication means AND.

$$\begin{aligned}
A^+ &= E_{10}D + E_{15}N,\\
B^+ &= E_0D + E_5N + E_5D + E_{10}N,\\
C^+ &= E_0N + E_5D + E_{10}N,\\
P &= Q(E_0 + E_5 + E_{10} + E_{15} + E_{20}) + D(E_{15} + E_{20}) + NE_{20},\\
R_5 &= Q(E_5 + E_{15}) + DE_{20},\\
R_{10} &= Q(E_{10} + E_{15}),\\
R_{20} &= QE_{20}.
\end{aligned}$$

These are the combinational transaction results. The Verilog stores them in output registers when a valid coin is accepted. Reset clears the outputs and selects S_0; otherwise, an idle input leaves the registers unchanged.

1.6 FSM Schematic and Verilog Implementation

The controller consists of input conditioning, combinational next-state and output logic, a resettable state register, and registered transaction outputs. The state register feeds the current balance back to the combinational logic.



Registers: clock = `clk`; asynchronous reset = `reset`; enable = `valid_coin`

Figure 2: Functional schematic of the supplied board implementation.

The board uses a 100 MHz clock, so one clock period is $T = 1/(100 \times 10^6) = 10$ ns. A 10 ms debounce interval requires

$$(100 \times 10^6)(10 \times 10^{-3}) = 1{,}000{,}000 \text{ clock cycles.}$$

A 20-bit counter is sufficient because $2^{20} = 1{,}048{,}576$. Each switch is synchronized and debounced, then its rising edge produces a single-cycle coin pulse. This counts a held switch only once. Only a pulse for exactly one coin enables a transaction.

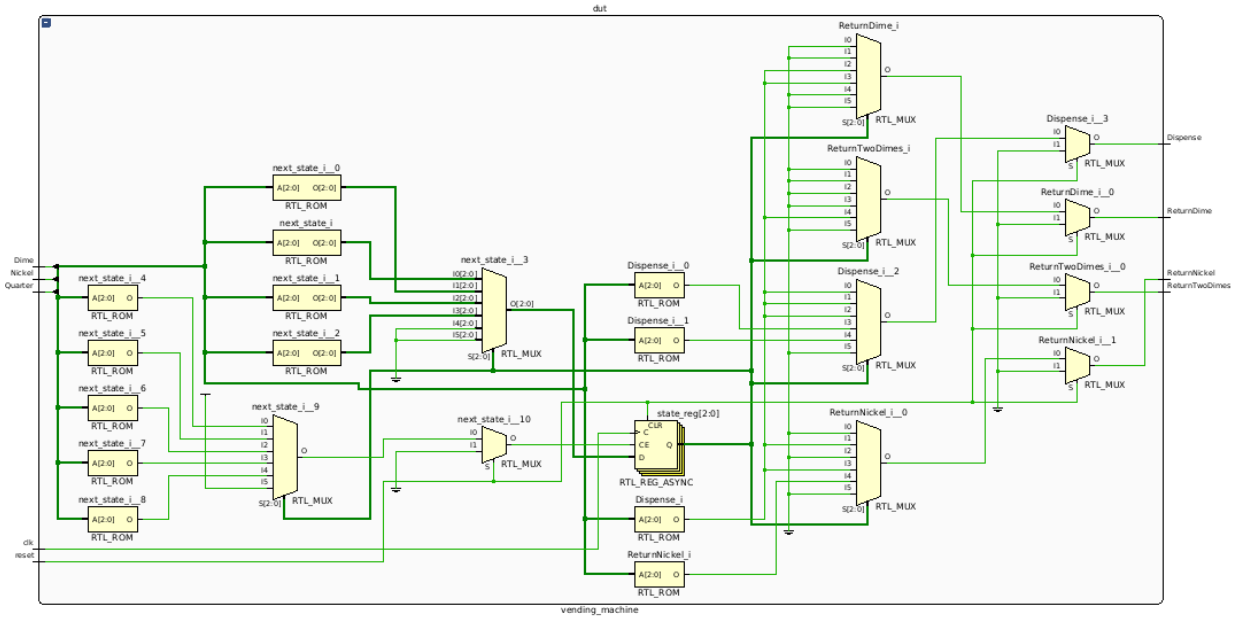


Figure 3: Supplied RTL schematic showing the combinational logic, state register, and vending-machine outputs.

SW0 is reset; SW1, SW2, and SW3 select a nickel, dime, and quarter. LED0–LED3 retain the transaction result until the next accepted coin or reset. LED4–LED8 indicate the balance with one active LED. The testbench uses `DEBOUNCE_CYCLES=3` and a 10 ns clock period.

2. Compiled Verilog and Console Results

XSim loaded the design and reported five passing tests in the 1000 ns capture. The packaged Verilog also compiled without errors in Icarus Verilog; all eight tests passed.

```
INFO: [USF-XSim-98] *** Running xsim
   with args "vending_machine_tb_behav -key {Behavioral:sim_1:Functional:vending_machine_tb} -tclbatch {vending_machine_tb.tcl} -log {simulate.log}"
INFO: [USF-XSim-8] Loading simulator feature
Time resolution is 1 ps
source vending_machine_tb.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#     send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform Configuration' or type 'create_wave_config' in the TCL console."
#   }
# }
run 1000ns
PASS test 1 | StateLED=00001
PASS test 2 | StateLED=00010
PASS test 3 | StateLED=00001
PASS test 4 | StateLED=00100
PASS test 5 | StateLED=00001
INFO: [USF-XSim-96] XSim completed. Design snapshot 'vending_machine_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
```

Figure 4: Supplied XSim console with the loaded design and passing test results.

3. Simulation Waveforms and Test Vectors

The waveform shows coin inputs, transaction outputs, state LEDs, and the error counter. The testbench holds and releases each coin for eight clock cycles each.

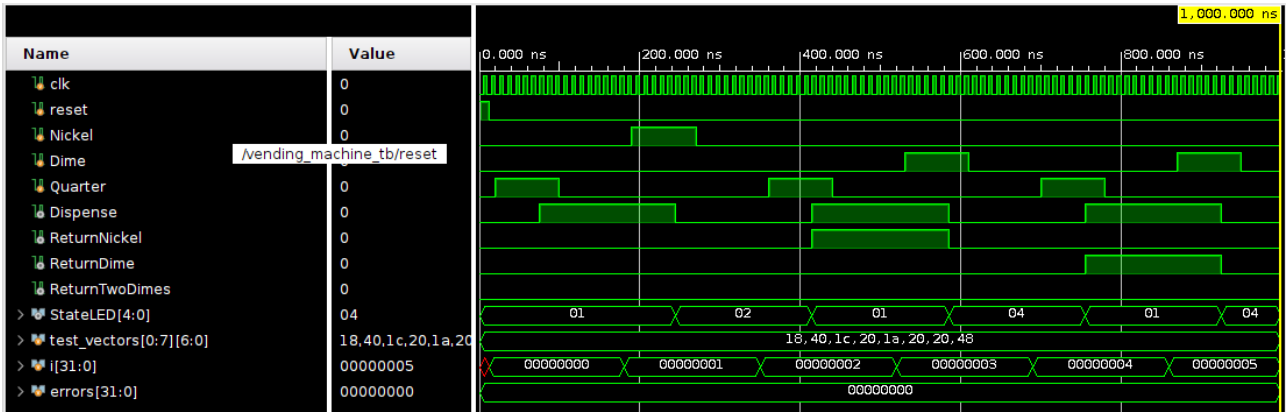


Figure 5: Supplied 0–1000 ns waveform showing the early transactions and zero errors.

Test	Vector	Balance before	Coin	Expected (P, R_5, R_{10}, R_{20})	Balance after
1	0011000	0	Quarter	1000	0
2	1000000	0	Nickel	0000	5
3	0011100	5	Quarter	1100	0
4	0100000	0	Dime	0000	10
5	0011010	10	Quarter	1010	0
6	0100000	0	Dime	0000	10
7	0100000	10	Dime	0000	20
8	1001000	20	Nickel	1000	0

4. Verification Summary

The simulation verified balance accumulation, dispensing at 25 cents, and a return to S_0 after each purchase. A quarter dispensed with no change. A nickel followed by a quarter returned a nickel; a dime followed by a quarter returned a dime. Two dimes followed by a nickel dispensed with no change. The waveform shows the input conditioning delay and held outputs. All eight supplied vectors matched their expected outputs with zero failed tests.

Separate source files: `vending_machine.v`, `vending_machine_tb.v`, and `test_vector.tv`.